CARBONHARDWARE

Progress, Technical Status, and Next-Step Handoff Report

Prepared for transfer to another AI assistant

Report date: 25 October 2026
Repository: Elmer015/CarbonHardware
Working directory: D:/COLLEGE/SEM 5/Research Method/CarbonHardware
Current scope: embodied carbon, operational energy/carbon, CCI, uncertainty, frontend/backend integration

Important scope decisions: database work is intentionally postponed; further STEC development is intentionally on hold. This report describes the current implementation and the exact remaining work without treating experimental estimates as measured LCA results.

1. Executive summary

CarbonHardware is a research application for estimating processor carbon metrics. The current system has a React/Vite frontend, a Node.js/Express backend, and a Python ML/calculation engine. Existing processors are selected from CarbonSet; unsupported hardware can be entered as a custom processor. The backend now loads a local OWID/Ember lifecycle electricity-intensity snapshot and calculates operational carbon from region and analysis year rather than asking the user to enter an arbitrary grid factor.

The main end-to-end path is now available: select or define hardware, provide a usage scenario, select a usage region and year, calculate embodied carbon with the experimental Random Forest model, calculate operational energy and operational carbon, and calculate CCI from embodied plus operational carbon. Existing processors also receive a CarbonSet min/mean/max uncertainty summary.

> **Current overall status:** the primary embodied + operational carbon flow is implemented and tested. The largest remaining work is uncertainty propagation for operational/total carbon, stronger region-selection UX and validation, methodology refinement, and research-grade evidence/provenance. Database persistence and STEC expansion remain intentionally deferred.

2. Architecture and data flow

Frontend: React/Vite form and result view in `frontend/src`. It calls the backend at `VITE_API_URL`, defaulting to `http://localhost:3000`.

Backend: Node.js/Express API in `backend/src`. It loads CarbonSet and the grid-intensity CSV at startup, validates requests, calls the Python model for embodied predictions, and performs operational/metric calculations.

Python engine: `ml-engine` contains embodied model integration, uncertainty, CCI, operational-carbon reference logic, and STEC. The Node backend invokes the embodied and STEC Python functions through a subprocess bridge.

```
Frontend form
-> POST /api/estimates/embodied
-> POST /api/estimates/operational-carbon
-> POST /api/estimates/uncertainty (existing processors)
-> POST /api/estimates/cci with embodied + operational total
-> frontend result card
```

3. Completed work

3.1 Hardware input and CarbonSet integration

• Existing processor mode uses CarbonSet records loaded by the backend. The frontend no longer needs to calculate model features itself.

• Custom processor mode accepts processor name, process size, TDP, die size, number of dies, and average die area.

• Backend validation rejects missing, negative, non-finite, and invalid integer inputs.

• CarbonSet contains 1320 rows and is deduplicated to 1272 unique processor names at load time.

• Embodied model features are exactly: Process Size (nm), TDP (W), DieSizeValue, #dies, and Avg Die Area.

3.2 Embodied carbon model integration

The Node endpoint `POST /api/estimates/embodied` invokes the experimental Random Forest joblib artifact through Python. The response is explicitly labelled as a model prediction, not a measured or reported LCA value. Usage duration and utilization do not change embodied carbon because they are not features of this model; they affect operational energy/carbon instead.

```
POST /api/estimates/embodied
{
"inputMode": "existing",
"processorName": "AMD Ryzen Threadripper 1900X"
}
```

```
Example model result: approximately 5.54065 kg CO2e
```

```
POST /api/estimates/embodied
{
"inputMode": "custom",
"processorName": "Custom Research CPU",
"processSizeNm": 7,
"tdpW": 250,
"dieSizeMm2": 400,
"dies": 2,
"avgDieAreaMm2": 200
}

Example model result: approximately 9.12919 kg CO2e
```

3.3 Operational energy

Operational energy is calculated with a transparent TDP-based power proxy:

```
electricity_kwh = tdp_w / 1000 * hours_per_day * days_per_year * usage_years * utilization
```

Example: 180 W, 8 hours/day, 365 days/year, 5 years, and 70% utilization produce 1,839.6 kWh. TDP is an assumption/proxy, not a measurement of actual wall power.

3.4 OWID/Ember grid-intensity integration

The local snapshot is stored at `data/raw/environmental/owid-lifecycle-carbon-intensity.csv`, with metadata beside it. The loader validates the expected columns Entity, Code, Year, and Carbon intensity, loads 6,332 records, and converts gCO2e/kWh to kgCO2e/kWh by dividing by 1,000. It does not silently use the nearest year or a default region.

The source is lifecycle-based electricity intensity from Ember via Our World in Data. It includes upstream, supply-chain, manufacturing, and other greenhouse-gas lifecycle stages. Therefore, outputs must be labelled lifecycle-based location-based operational carbon, not direct generator-only emissions.

```
GET /api/grid-intensity?region=Indonesia&year=2024

{
"region": "Indonesia",
"year": 2024,
"gramsCo2ePerKwh": 680.25,
"kgCo2ePerKwh": 0.68025
}
```

3.5 Operational carbon API

The endpoint `POST /api/estimates/operational-carbon` combines operational energy with the region/year grid lookup:

```
POST /api/estimates/operational-carbon
{
"tdpW": 180,
"hoursPerDay": 8,
"usageYears": 5,
"utilization": 0.7,
"daysPerYear": 365,
"region": "Indonesia",
"analysisYear": 2024
}

energy = 1,839.6 kWh
grid factor = 0.68025 kgCO2e/kWh
operational carbon = 1,251.3879 kgCO2e
```

If a region/year is absent, the API returns HTTP 404 with an explicit message. For example, Indonesia 2025 is unavailable in this snapshot, while Indonesia 2024 is available.

3.6 CCI

The frontend now sends total carbon as embodied plus operational carbon. The CCI equation is:

```
CCI = (embodied_carbon_kg_co2e + operational_carbon_kg_co2e) / completed_compute_work
```

The workload unit must be explicit, such as completed training run, inference request, million tokens, or another reproducible unit. CCI is only comparable when workload definition, boundary, and analysis period are equivalent.

3.7 Uncertainty

Existing processors use CarbonSet Emb_Carbon_Min, Emb_Carbon_Mean, and Emb_Carbon_Max as an evidence-bounded embodied range. Sample-based uncertainty is also supported by the Python engine through empirical quantiles and is labelled not calibrated coverage. Custom processors currently do not have a validated uncertainty range.

3.8 STEC status

A location-based die-level STEC calculator exists in `ml-engine/carbon/stec.py`. It implements the workbook contract and includes node parameters, the explicit 22 nm to 20 nm proxy, optional yield correction, input validation, and metadata. The implemented boundary is cradle-to-gate die manufacturing. Further STEC work is on hold by decision and was not expanded in this integration.

4. Validation completed

Validation	Result
Backend integration tests	Passed: 1 test, 0 failures. Covers processor lookup, grid lookup, operational carbon, unavailable year, and CCI validation.
Frontend production build	Passed with Vite.
Grid dataset loader	Passed: 6,332 records loaded and Indonesia 2024 lookup verified.
Operational carbon example	Passed: 1,839.6 kWh and 1,251.3879 kgCO2e for the Indonesia 2024 example.
Backend runtime	Verified on localhost:3000 with the new operational-carbon and grid-intensity routes.
Dependency audit	Earlier npm audit reported 0 vulnerabilities for the backend dependency set.

5. Explicitly incomplete or not implemented

Area	Current state and impact
Operational uncertainty	Not implemented. No propagation for TDP, runtime, utilization, lifetime, grid factor, or regional/year variation.
Total-carbon uncertainty	Not implemented. Current uncertainty covers embodied CarbonSet range only and must not be presented as a total-carbon interval.
Custom uncertainty	Not available because custom processors lack a source-backed distribution or interval.
Region UX	Region is currently a text input. A dropdown/search control backed by /api/grid-intensity/regions is not yet wired.
Market-based attribution	Not implemented. Current grid result is lifecycle-based location-based.
Database	Intentionally postponed. No estimate/scenario/history persistence is implemented.
STEC expansion	On hold. No full workbook pipeline, multi-fab/chiplet allocation, or final manufacturing provenance integration.
Cooling/PUE/overhead	Not modeled in the current operational formula. If added, it must be an explicit scenario input or documented assumption.

Area	Current state and impact
Research-grade data governance	Snapshot is copied locally, but automated update/version management and final citation review remain.

6. Known boundaries and interpretation rules

• Embodied output is an experimental model prediction, not measured LCA.

• Operational energy uses TDP as a power proxy; actual power measurement is not available.

• Operational carbon uses the processor usage location, while STEC uses manufacturing/fab location. These locations must not be conflated.

• Lifecycle grid intensity is not the same as direct stack emissions.

• CCI requires a defined completed-work unit; runtime alone is not automatically an equivalent workload.

• Embodied, operational, STEC, e-waste, and uncertainty quantities must remain separate and carry unit/boundary metadata.

• No silent defaults should be introduced for missing grid data, unknown regions, unsupported years, or custom uncertainty.

7. Recommended step plan

The following sequence is recommended for the next AI or developer. It keeps the current scope stable and postpones database/STEC as requested.

Priority	Step	Deliverable / acceptance criteria
1	Improve grid-region UX	Load /api/grid-intensity/regions into a searchable dropdown; display available-year errors clearly; retain selected region/year in result metadata.
2	Separate API calculation contract	Add a single scenario endpoint or service that returns embodied, energy, operational, total carbon, CCI, and provenance consistently. Keep existing endpoints backward-compatible.
3	Operational uncertainty design	Agree on distributions/ranges for TDP proxy, utilization, runtime, lifetime, days/year, and grid intensity before coding. Document whether results are scenario ranges or probabilistic intervals.
4	Implement uncertainty propagation	Use deterministic sensitivity or Monte Carlo only after inputs and sample count/seed are specified. Label coverage and calibration honestly. Add tests for interval ordering and reproducibility.
5	Improve CCI contract	Validate workload unit and boundary, add explicit total-carbon fields, and prevent mixing embodied-only and embodied-plus-operational CCI without a boundary marker.
6	Add evidence/provenance checks	Record dataset snapshot, source URL, download date, metric, basis, original unit, converted unit, region, and year in every operational result.
7	Add frontend result clarity	Show separate cards for embodied, operational energy, operational carbon, total carbon, CCI, and uncertainty; show assumptions such as TDP proxy and lifecycle grid basis.
8	Testing hardening	Add tests for missing CSV, malformed row, unsupported region/year, case-insensitive region lookup, zero/invalid inputs, custom processor flow, and CCI boundary mismatch.
9	Research review	Verify OWID/Ember citation and snapshot date, compare selected values against the source, and obtain methodology approval before using outputs as final research claims.
10	Only later: database	Persist processor/scenario/result metadata after the calculation contract is stable.
11	Only later: STEC	Resume after fab provenance, allocation, chiplet rules, and workbook validation requirements are finalized.

8. Suggested handoff prompt for another AI

Use the following context when continuing work:

```
You are continuing CarbonHardware in D:/COLLEGE/SEM 5/Research Method/CarbonHardware.

Implemented: React/Vite frontend; Node/Express backend; CarbonSet existing/custom processor
modes; experimental joblib embodied prediction; TDP-based operational energy; local OWID/Ember
lifecycle grid-intensity snapshot; region/year grid lookup; operational carbon endpoint;
frontend integration; CCI using embodied + operational carbon; embodied CarbonSet uncertainty;
tests and frontend build.

Do not start database work unless explicitly requested. Do not expand STEC unless explicitly
requested. Preserve explicit units, boundaries, source metadata, and no-silent-fallback errors.
Operational carbon formula: electricity_kwh * grid_intensity_kg_co2e_per_kwh. Current main gap:
operational and total-carbon uncertainty, region dropdown UX, stronger provenance and tests.

Important files: backend/src/services/grid-intensity.service.js,
backend/src/services/carbon.service.js, backend/src/controllers/estimate.controller.js,
backend/src/controllers/grid.controller.js, backend/src/routes/grid.routes.js,
frontend/src/components/HardwareForm.jsx, frontend/src/pages/EstimateCarbon.jsx,
data/raw/environmental/owid-lifecycle-carbon-intensity.csv, README.md, backend/README.md.
```

9. File inventory

File or folder	Purpose
backend/src/services/grid-intensity.service.js	CSV schema validation, loading, unit conversion, lookup, metadata.
backend/src/services/carbon.service.js	Operational energy and operational carbon calculations.
backend/src/controllers/estimate.controller.js	Embodied, operational energy/carbon, STEC, CCI, uncertainty request handling.
backend/src/controllers/grid.controller.js	Grid lookup and region-list HTTP controllers.
backend/src/routes/grid.routes.js	Grid intensity routes.
frontend/src/components/HardwareForm.jsx	Processor, operational scenario, region/year, and CCI inputs.
frontend/src/pages/EstimateCarbon.jsx	Frontend API orchestration and result presentation.
data/raw/environmental/	Controlled local OWID/Ember CSV and metadata snapshot.
backend/test/api.test.js	Backend integration tests for current API flow.
README.md and backend/README.md	User/developer documentation and API examples.
docs/carbonhardware-progress-handoff-2026-10-25.pdf	This handoff report.

10. Final status

Ready now: a user can run the frontend and backend, select an existing or custom processor, provide usage assumptions, select a region/year, calculate embodied carbon, calculate operational energy and lifecycle operational carbon, and calculate CCI using the combined total. Backend and frontend validation passed.

Not ready to claim as final research methodology: operational/total uncertainty, market-based attribution, measured power, full cooling/PUE treatment, automated dataset governance, database persistence, and expanded STEC research workflow.

Recommended immediate next action: implement the region dropdown and agree on the uncertainty methodology before adding more calculations. This avoids presenting a precise-looking total without a defensible uncertainty model.

ADDENDUM - UNCERTAINTY IMPLEMENTATION UPDATE

25 October 2026 | Scope: Option A total carbon and custom processor uncertainty

Decision implemented: PUE and market-based attribution remain intentionally out of scope. The work in this update is limited to uncertainty.

1. Completed in this update

• Added POST /api/estimates/uncertainty/total for Option A interval addition.

• The endpoint validates embodied and operational lower/central/upper values, adds corresponding bounds, and optionally derives an interval CCI.

• Added POST /api/estimates/uncertainty/custom for custom processor embodied sensitivity.

• The custom endpoint runs the embodied model at min, central, and max values for process size, TDP, die size, number of dies, and average die area.

• Custom results are explicitly labelled three-point boundary sensitivity; not calibrated coverage. They are not presented as a calibrated confidence interval.

• Added backend integration coverage for total carbon uncertainty, CCI uncertainty, and invalid custom uncertainty requests.

• Updated README.md and backend/README.md with contracts, examples, limitations, and interpretation rules.

2. Option A total-carbon method

The selected method is independent interval addition. It requires a source-backed or scenario-backed interval for each component; the backend does not invent operational bounds.

```
Total lower = embodied lower + operational lower
Total central = embodied central + operational central
Total upper = embodied upper + operational upper
```

Example input:

```
{
  "embodied": {"lowerValue": 1, "centralValue": 2, "upperValue": 3},
  "operational": {"lowerValue": 10, "centralValue": 20, "upperValue": 30},
  "completedComputeWork": 5,
  "workloadUnit": "training run"
}
```

Result: total carbon = 11 / 22 / 33 kgCO2e. Interval CCI = 2.2 / 4.4 / 6.6 kgCO2e per training run.

3. Custom processor uncertainty method

Custom processor uncertainty cannot reuse CarbonSet min/mean/max because a custom processor has no CarbonSet range. The new endpoint accepts explicit input bounds:

```
{
  "processSizeNm": {"min": 7, "central": 7, "max": 8},
  "tdpW": {"min": 220, "central": 250, "max": 280},
  "dieSizeMm2": {"min": 350, "central": 400, "max": 450},
  "dies": {"min": 2, "central": 2, "max": 2},
  "avgDieAreaMm2": {"min": 175, "central": 200, "max": 225}
}
```

The model is run three times: all minimum inputs, all central inputs, and all maximum inputs. The output interval uses the minimum and maximum predictions, while the central value is the central-input prediction. This is a transparent sensitivity

method, not a probability distribution.

4. Still incomplete after this update

Area	Status after update
Operational interval input UX	Not yet exposed as dedicated min/central/max fields in the frontend. The total endpoint is ready, but the caller must supply defensible operational bounds.
Custom uncertainty UI	Backend endpoint is ready; frontend does not yet expose all five custom hardware ranges.
Operational uncertainty derivation	Not automatically derived from TDP, utilization, lifetime, or grid data. This is intentional to avoid invented uncertainty.
Total uncertainty integration in main result card	Current frontend still shows the existing embodied uncertainty flow; total interval display requires wiring the new endpoint.
PUE and market-based attribution	Not implemented by decision.
Database and STEC expansion	Still postponed/on hold.

5. Verification

Backend test result: 1 test passed, 0 failed. Frontend production build: passed. The new total uncertainty test verifies total bounds and interval CCI; invalid custom bounds are rejected with HTTP 400.

6. Recommended next steps

• Add frontend controls for operational lower/central/upper scenario bounds and show their assumptions.

• Add frontend controls for custom processor min/central/max hardware inputs.

• Wire total uncertainty and interval CCI into the main result card.

• Add tests for non-monotonic model predictions, missing operational bounds, and boundary/lifecycle mismatches.

• Only after these are stable, consider a documented Monte Carlo method; do not call empirical ranges confidence intervals without calibration.

CORRECTION / FINAL UNCERTAINTY UI STATUS

This page supersedes the earlier addendum sentence stating that the custom uncertainty UI was not exposed.

The frontend now exposes min/max fields for process size, TDP, die size, number of dies, and average die area when Custom processor mode is selected. On submit, it calls POST /api/estimates/uncertainty/custom and displays the resulting custom sensitivity range.

The total-carbon Option A endpoint is implemented and tested at POST /api/estimates/uncertainty/total. Its dedicated frontend controls for operational lower/central/upper bounds and total interval display are still pending because operational bounds must be supplied from defensible scenario assumptions rather than invented defaults.

Final interpretation: custom embodied uncertainty is available through the UI as three-point boundary sensitivity; total-carbon uncertainty is available through the backend contract and is ready for frontend wiring once operational bounds are collected.